

CVX4PY: A modeling framework in Python for Disciplined Convex Programs

Arthita Ghosh, Hui Ding, Sayantan Sarkar

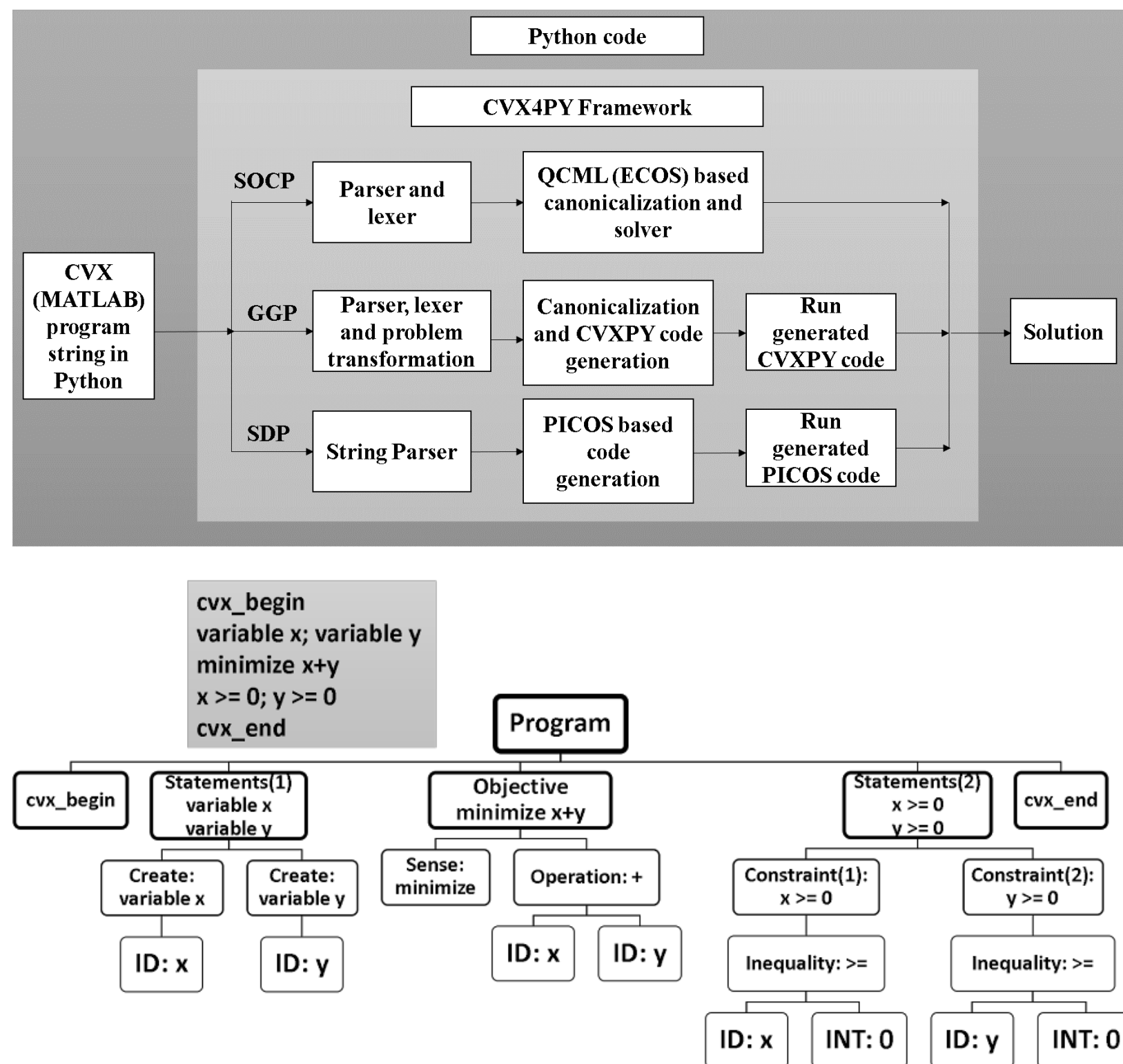
Objective

Solving Convex Programs in Python through packages like QCML (ECOS), CVXPY, CVXOPT, PICOS etc has the some limitations. To mitigate these problems we propose the CVX4PY framework that lets users solve Generalized Geomtric Programs (GGP), and some forms of Second Order Conic Programs (SOCP) and Semi Definite Programs (SDP) by defining the problems in CVX syntax. Specifically our contributions are:

- Defining the problem is easy and intuitive as CVX format is used
- Code written for MATLAB in CVX can directly be used in Python
- GGPs can be defined in their native, non-convex forms
- CVX4PY brings various packages with different pros and cons together in a single framework. It uses CVXPY to solve GGPs, ECOS to solve SOCPs and PICOS to solve SDPs.
- A library of toy examples and a case study of a real world problem.

Case study: Minimize power in the multicell coordination beamforming problem with imperfect channel information, subject to some minimum quality constraint. We propose a distributed SDP formulation (using primal decomposition) that can be solved using CVX4PY

CVX4PY Framework



Features

- **General:** Equality and inequality constraints, maximization and minimization, values of variable and objective
- **GGP:** Monomials, Posynomials, Generalized Posynomials, Dual Variables, feasibility, comments
- **SOCP:** Linear expressions, norm and sum functions, chained constraints, feasibility, comments
- **SDP:** Standard form SDP (real or complex)

Generalized Geometric Program

Denoting monomials by m , posynomials by p and real numbers by r , generalized posynomials (g.p.) as h , the rules for creating new g.p.s are as follows:

$$h = \max(h_1, \dots, h_k), \quad h = h_1^r, \quad h = p$$

Denoting t as a new variable, g.p.s can be transformed to valid GP constraints as follows:

$$h^r \leq m \rightarrow t^r \leq m, \quad h \leq t$$

$$\max(h_1, \dots, h_k) \leq m \rightarrow t \leq m, \quad h_1 \leq t, \dots, h_k \leq t$$

GP constraints can be canonicalized as follows, before replacing them with exponentiated variables that make the constraints convex:

$$m_1 = m_2 \rightarrow m_1/m_2 = 1$$

$$m_1 \leq m_2 \rightarrow m_1/m_2 \leq 1$$

$$m_1 \geq m_2 \rightarrow m_2/m_1 \leq 1$$

$$p_1 \leq m_1 \rightarrow p_1/m_1 \leq 1$$

$$m_1 \geq p_1 \rightarrow p_1/m_1 \leq 1$$

Examples

A GGP example	An LP example
<pre>from cvx4py import cvx4py string = """ cvx_begin gp variables x y z dual variable d maximize x*y*z d: x*y + y*z <= 100 (x*z + (z)*(y^0.5))^0.5 <= 50 max(x, x+z) <= 100 max(x, (x+2*z)^0.4) <= 100 x <= 10; y <= 20; z <= 30 cvx_end"""</pre>	<pre>from cvx4py import cvx4py a=14 string = """ cvx_begin variables x y minimize 2*x + 4*y 2*x + y >= a x + y >= 12 x + 3*y >= 18 10 >= x >= 0 y >= 0 cvx_end"""</pre>
<pre>p=cvx4py(string,0,locals()) soln=p.solveProblem(); print soln</pre>	<pre>p=cvx4py(string,0,locals()) soln=p.solveProblem(); print soln</pre>
A Robust LP example	An SDP example
<pre>c = np.array([[1],[2]]) A = np.array([[1,-1,0], [-1,0,-1]]); b = np.array([1,0,0]) a1 = np.array([[1],[1]]) ; b1 = 5 P1 = np.array([[1,2],[2,1]])</pre>	<pre>n = 3 C=cvx.matrix([[1,2,3],[2,9,0],[3,0,7]]) A1=cvx.matrix([[1,0,1],[0,3,7],[1,7,5]]) A2=cvx.matrix([[0,2,8],[2,6,0],[8,0,4]]) b1 = 11; b2 = 19</pre>
<pre>string = """ cvx_begin variables x(2) maximize c'*x subject to A'*x <= b a1'*x + norm(P1*x) <= b1 cvx_end"""</pre>	<pre>string = """ cvx_begin sdp variable X(n,n) symmetric minimize(trace(C*X)) subject to trace(A1*X) == b1 trace(A2*X) == b2 X >= 0 cvx_end"""</pre>

Example: Distributed, Robust Beamforming

• Total Transmit Power Minimization Problem

$$\begin{aligned} & \text{minimize} \quad \sum_{i,k} \mathbf{w}_{i,k}^H \mathbf{w}_{i,k} && \text{Minimize Power} \\ & \text{subject to} \quad \Gamma_{i,k}(\mathbf{h} + \mathbf{e}) \geq \gamma_{i,k} && \text{SINR QoS Constraint} \end{aligned}$$

$$\Gamma_{i,k}(\mathbf{h}) = \frac{|\mathbf{h}_{i,i,k}^H \mathbf{w}_{i,k}|^2}{\sum_{l \neq k} |\mathbf{h}_{i,i,k}^H \mathbf{w}_{i,l}|^2 + \sum_{j \neq i} \sum_{l=1}^{N_c} |\mathbf{h}_{j,i,k}^H \mathbf{w}_{j,l}|^2 + \sigma_{i,k}^2}$$

where $\mathbf{w}_{i,k} \in \mathbb{C}^{K \times 1}$ are the optimization variables. $\|\mathbf{e}_{i,j,k}\| \leq \epsilon_{i,j,k}$.

• Sub-Problem

$$\begin{aligned} & \text{minimize} \quad \sum_k \text{tr}(\mathbf{W}_{i,k}) \\ & \text{subject to} \quad \begin{bmatrix} \mathbf{X}_{i,i,k} + \alpha_{i,i,k} \mathbf{I} & \mathbf{X}_{i,i,k} \tilde{\mathbf{h}}_{i,i,k} \\ \tilde{\mathbf{h}}_{i,i,k}^H \mathbf{X}_{i,i,k} & c_{i,i,k} \end{bmatrix} \succeq 0 \\ & \quad \begin{bmatrix} \mathbf{X}_i + \alpha_{i,j,l} \mathbf{I} & \mathbf{X}_i \tilde{\mathbf{h}}_{i,j,l} \\ \tilde{\mathbf{h}}_{i,j,l}^H \mathbf{X}_i & d_{i,j,l} \end{bmatrix} \succeq 0 \\ & \quad \mathbf{W}_{i,k} \succeq 0 \\ & \quad \forall i = 1, \dots, N_c, k = 1, \dots, K \end{aligned}$$

where $\mathbf{W}_{i,k} \in \mathbb{C}^{K \times K}$ $\alpha_{i,j,k} \in \mathbb{R}$ are the optimization variables.

• Master Problem

$$\begin{aligned} & \text{minimize} \quad \sum_i \phi_i(\beta_i) \\ & \text{subject to} \quad \beta_i \geq 0 \\ & \quad \forall i = 1, \dots, N_c, k = 1, \dots, K \end{aligned}$$

where ϕ_i is the optimal value of sub-problems when β_i is given. The optimization variable is $\beta_i \in \mathbb{R}$.

• Experimental Results

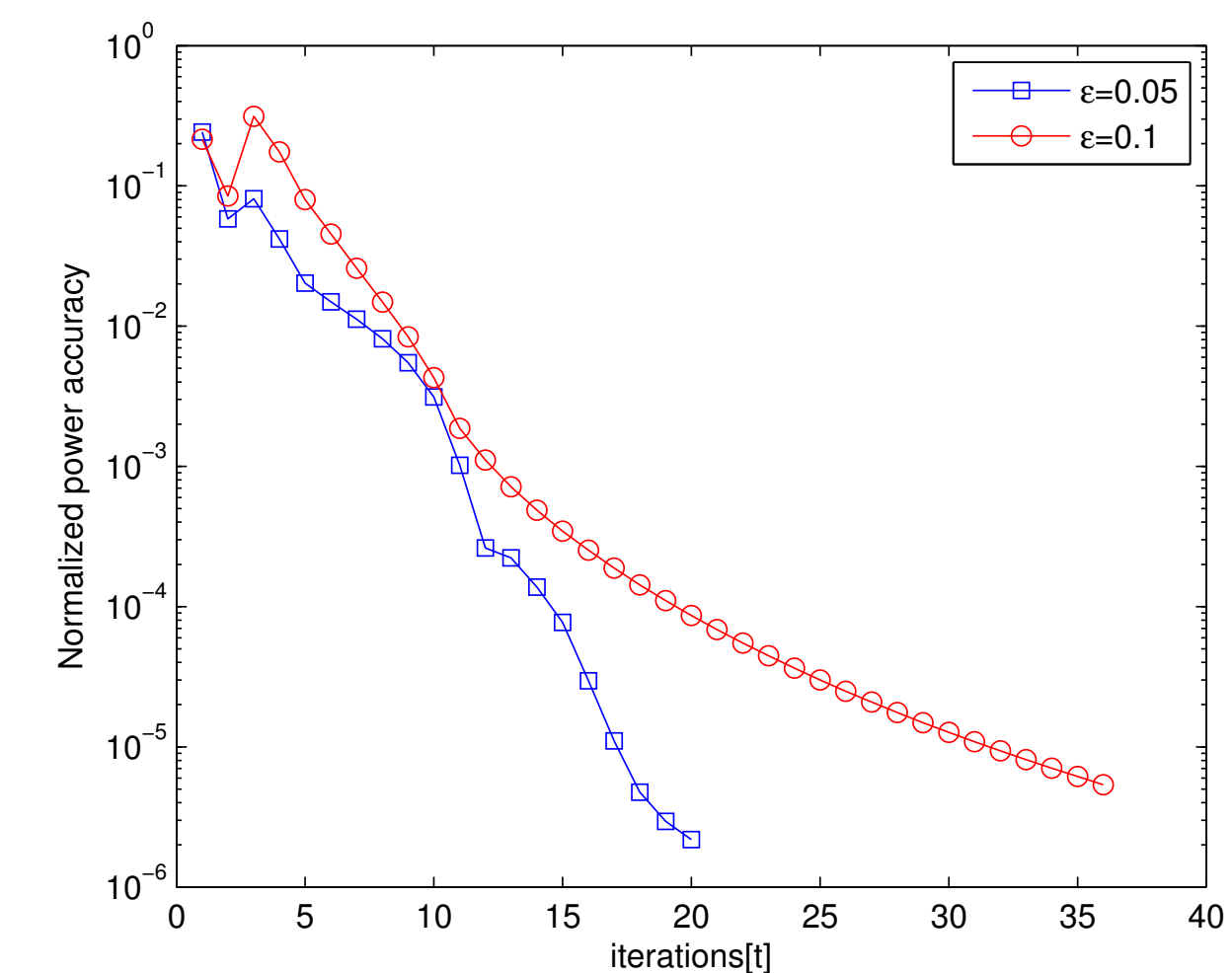


Figure 1: Convergence behavior of the proposed algorithm, $N_c=2$, $N_t=4$, $K=2$, $\text{SINR}=10\text{dB}$, ϵ is the error radius. The normalized power accuracy is defined as: $r(n) = \frac{|P_{dis}(n) - P_{cen}|}{P_{cen}}$.

Conclusion

Using CVX4PY Convex Problem modeling in Python is as accessible and intuitive as it is in MATLAB (using CVX).